

JSP

Java Server Pages

Generating Dynamic web content

Generating dynamic web content can be achieved through several technologies, mentioning few technologies like

CGI – Common Gateway Interface

ASP– Active Server Pages

Servlet

Though these technologies are working well in their domains, due to increased demands of client these technologies fall short of demands.

Let us have a glance over these technologies and problems encountered through these technologies.

ASP – Active Sever Pages

A Microsoft based scripting language that allows you to create HTML templates that insert dynamic content from the server (usually a database)

Problems

Runs on Microsoft Windows only

- Limited number of CPU architectures that can handle Windows

Windows is not as stable (opinion)

Servlet

Benefits:

- Conceivably faster since is partially compiled and hence it is
- Easier to “interpret.”
- Load the JVM once
- Platform independence

Problems:

not be appropriate for everyone;

no separation between content and presentation

Idea

Use regular HTML for most of page
Mark servlet code with special tags

Need ?

To meet current requirements, we need a technology which

- Works on any webserver or application server
- Separates the application logic from the presentation
- Allowing fast development and testing
- Simplifying the process of developing interactive web-based applications

The solution is **JSP** – **Java Server Pages**

The need for JSP

- With Servlets,it is easy to
 - Read form data
 - Read Http Request Headers
 - Use Cookies and session Tracking
 - Share data among servlets
 - Remember data between requests
- But, sure it is a pain to
 - use println to generate html
 - Maintain that html

What is JSP

A JSP page is a text based document that describes how to process a request to create a response.

Advantages of JSP

- Separation of content and display logic
- Write once run anywhere
- Reuse of components and tag libraries
- Recompile automatically
- Support for scripting and actions

Actions permit the encapsulation of useful functionality in a convenient form that can be used by tools. Scripts, glue together this functionality in a per-page manner

- Web access for N-tier enterprise application architecture

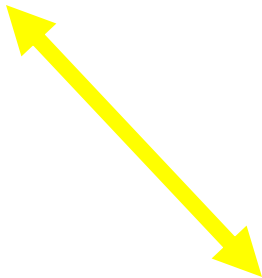
What is a Java Server Page

- A JSP is a simple text file consisting of HTML or XML content along with JSP elements.
- A JSP combines Java code and template HTML in a single file.
- This is similar to the way PHP works.
- Scripting elements are used to provide dynamic pages

What is JSP?

- Mostly HTML page, with extension .jsp
- Include JSP tags to enable dynamic content creation
- Translation: JSP → Servlet class
- Compiled at Request time
(first request, a little slow)
- Execution: Request → JSP Servlet's service method

What is a JSP?

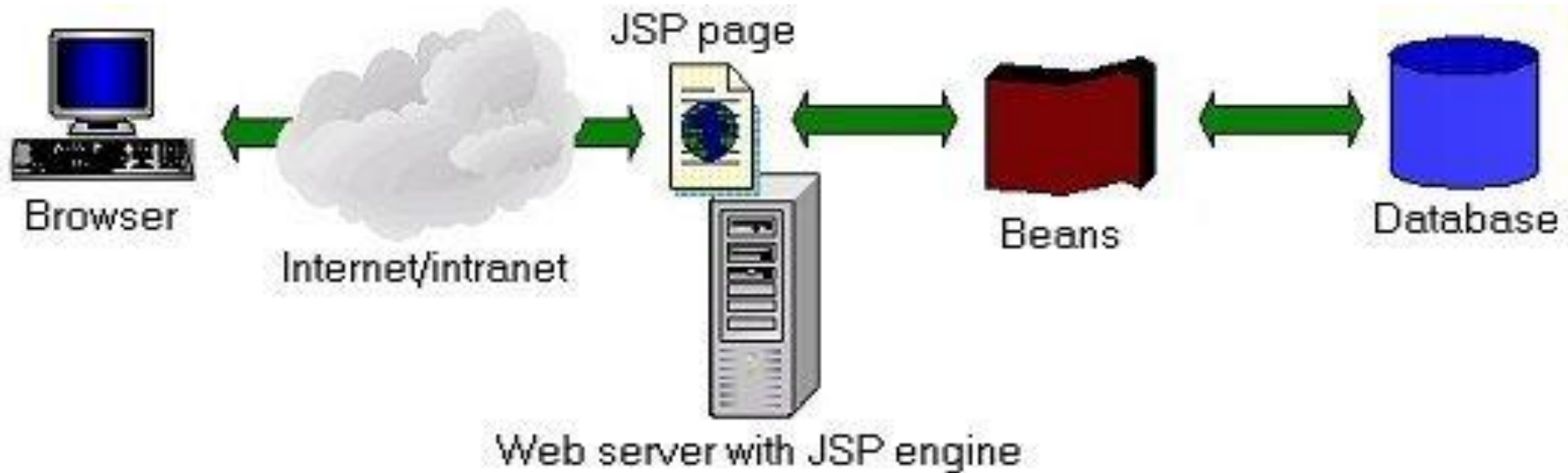


```
<html>
  <body>
    <jsp:useBean.../>
    <jsp:getProperty.../>
    <jsp:getProperty.../>
  </body>
</html>
```

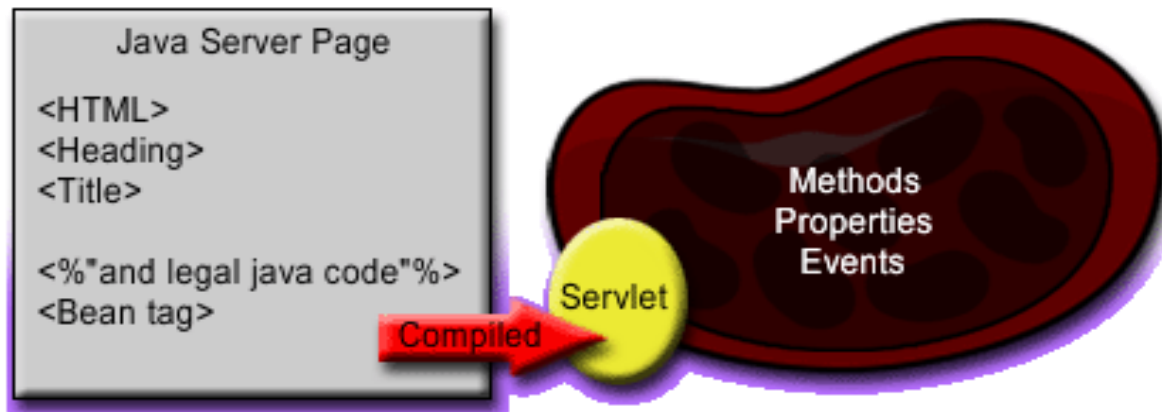
Advantages

- Code -- Computation
- HTML -- Presentation
- Separation of Roles
 - Developers
 - Content Authors/Graphic Designers/Web Masters
 - Supposed to be cheaper... but not really...

JSP Model



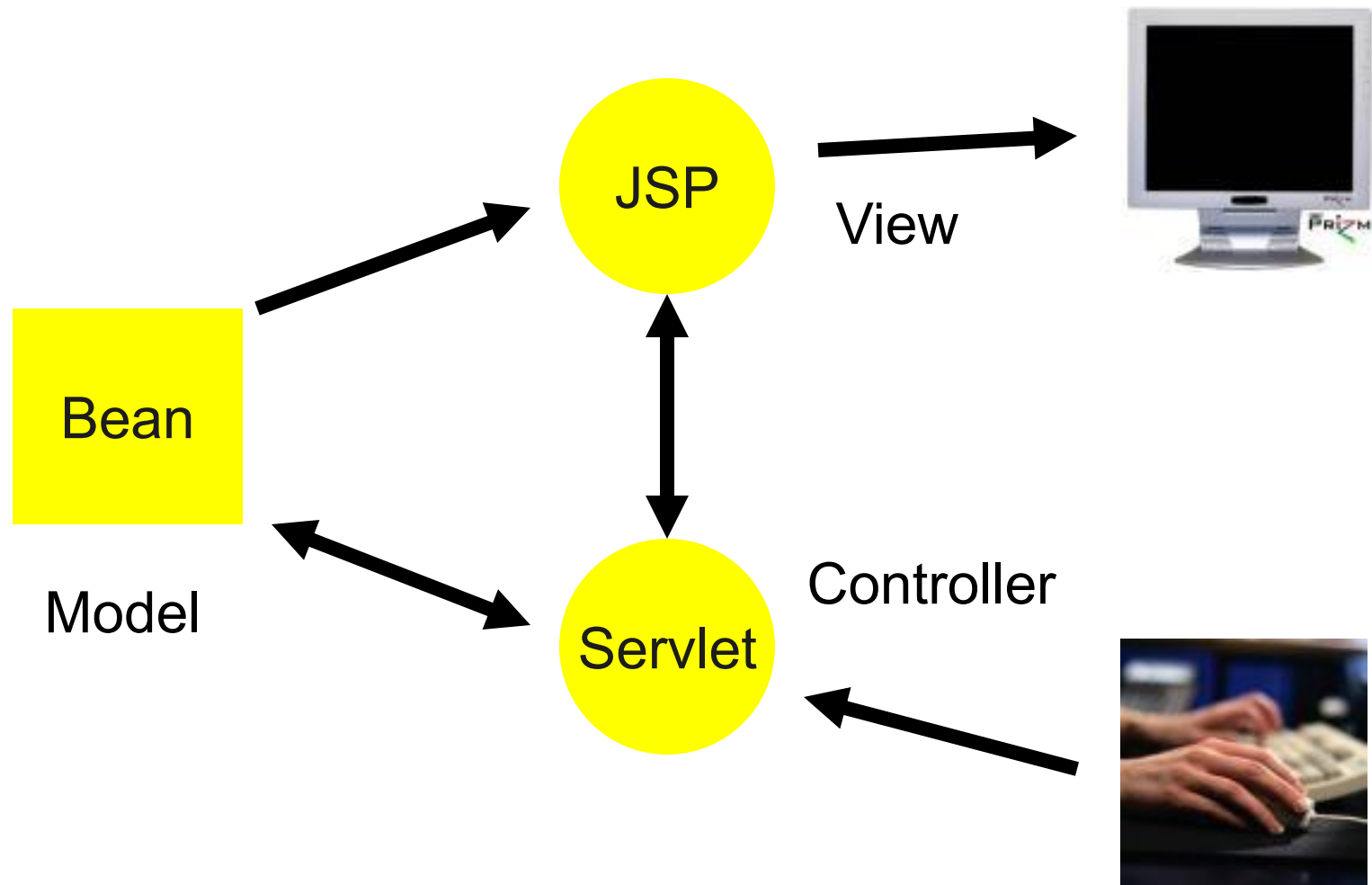
JSP Contents



Model-View-Controller

- A Design Pattern
- Controller -- receives user interface input, updates data model
- Model -- represents state of the world (e.g. shopping cart)
- View -- looks at model and generates an appropriate user interface to present the data and allow for further input

Model-View-Controller



Pure Servlet

```
public class OrderServlet ... {  
    public void doGet(...) {  
        if(isOrderValid(req)) {  
            saveOrder(req);  
        }  
        ...  
        out.println("<html><body>");  
        ...  
    }  
    private void isOrderValid(...) {  
        ...  
    }  
    private void saveOrder(...) {  
        ...  
    }  
}
```

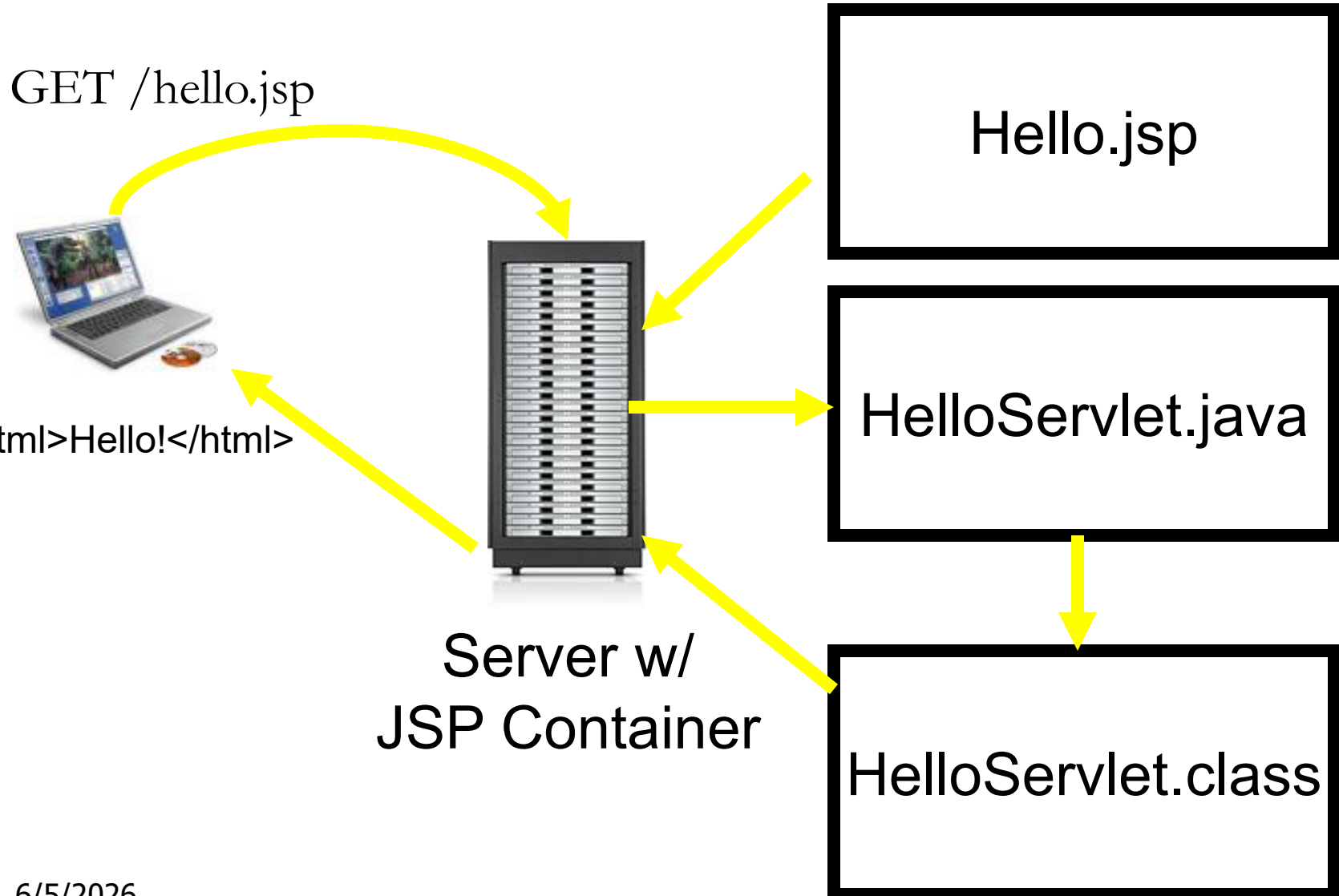
Servlet

JSP

Java Bean

```
Public class OrderServlet ... {  
    public void doGet(...) {  
        ...  
        if(bean.isOrderValid(...)) {  
            bean.saveOrder(req);  
        }  
        ...  
        forward("conf.jsp");  
    }  
}  
  
<html>  
  <body>  
    <c:forEach items="${order}">  
      ...  
    </c:forEach>  
  </body>  
</html>  
  
isOrderValid()  
saveOrder()  
-----  
private state
```


JSP Big Picture



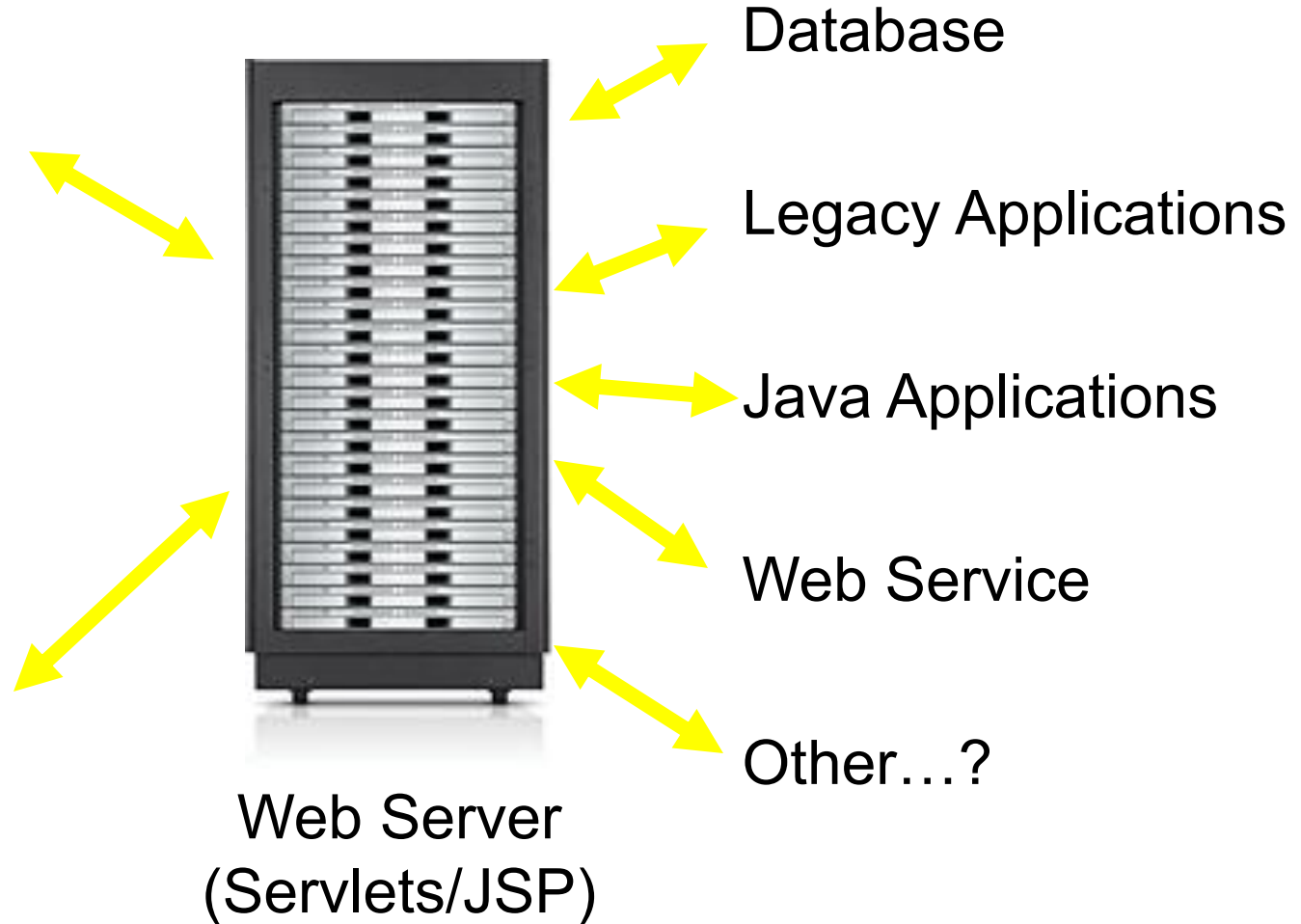
Big Picture – Web Apps



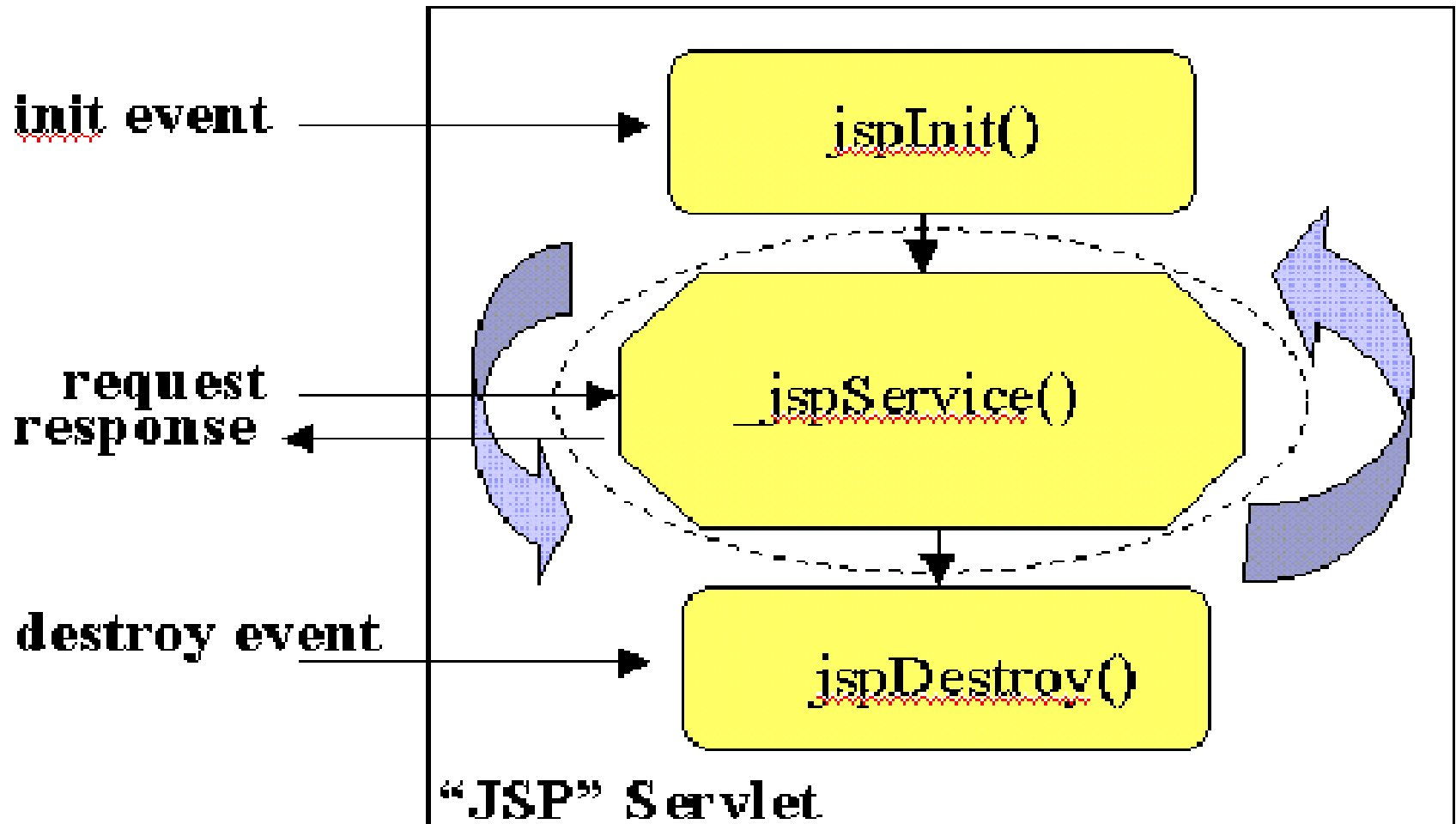
End User #1



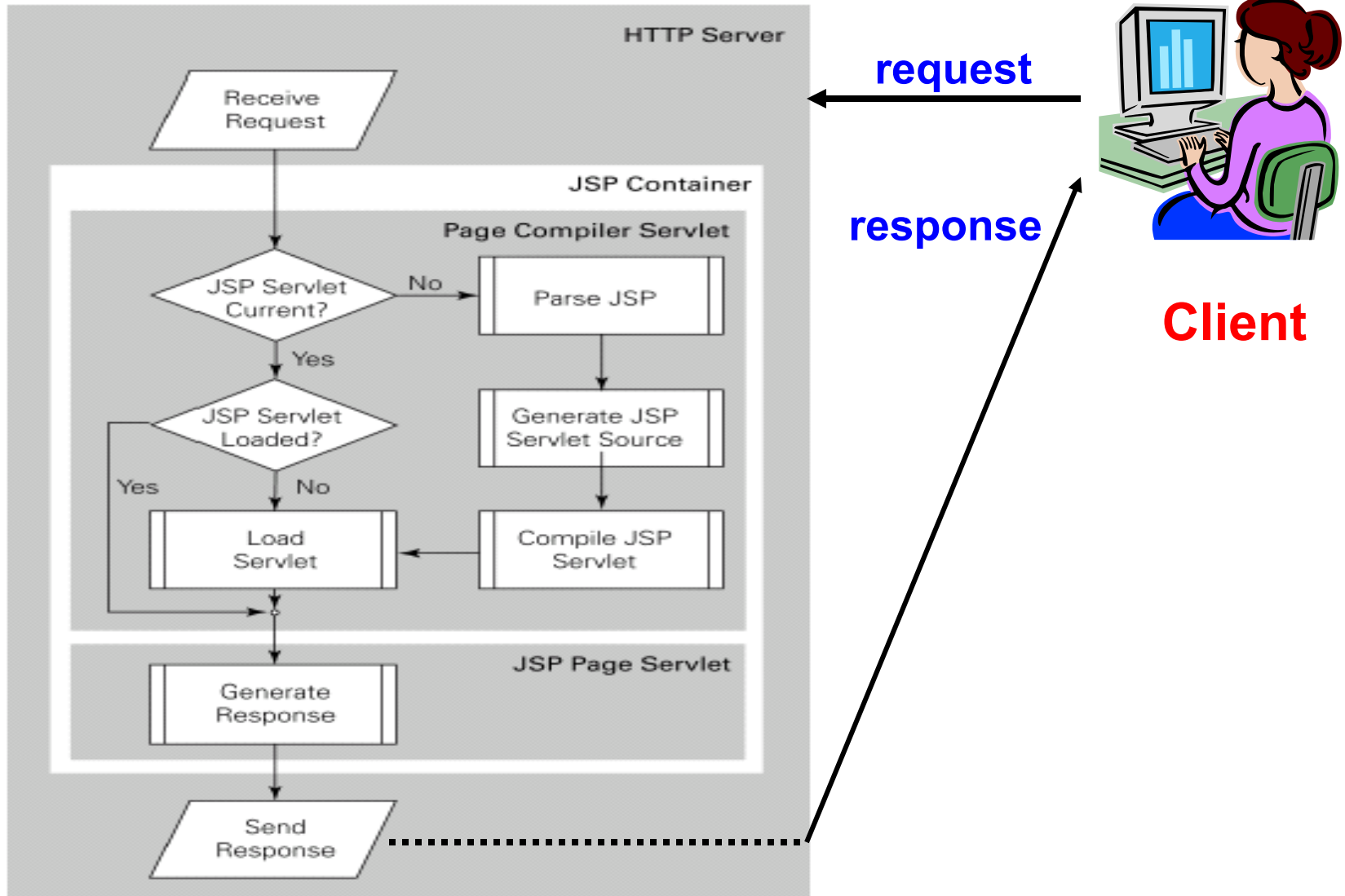
End User #2



JSP Life Cycle



JSP Page Processing



Invoking Dynamic Code (from JSPs)

- Call Java Code Directly
(Expressions, Declarations, Scriptlets)
- Call Java Code Indirectly
(Separate Utility Classes, JSP calls methods)
- Use Beans
(jsp:useBean, jsp:getProperty, jsp:setProperty)
- Use MVC architecture (servlet, JSP, JavaBean)
- Use JSP expression Language
(shorthand to access bean properties, etc)
- Use custom tags
(Develop tag handler classes; use xml-like custom tags)

Connection with servlets

- Each Java server page is compiled into a servlet before it can be used
- This is normally done when the first request is made so there could be a short wait.
- However, JSP's can be precompiled so there is no wait.



Translated servlets

- You can examine the source code produced by the JSP translation process.
- There is a directory called `work` in the main `tomcat` directory where you can find the source code.
- Note that the **`_jspService`** method corresponds to the servlet service method (which is called by **`doGet`** or **`doPost`**)

JSP Syntax

JSP Syntax - All

Comments

Output Comment

Hidden Comment

Scripting Elements

Declaration

Expression

Scriptlet

Directives

Page Include

Taglib

Actions

<jsp:forward>

<jsp:include>

<jsp:useBean>

<jsp:setProperty>

<jsp:getProperty>

<jsp:plugin>

Built In Objects

request response

application page

pageContextout

session exception

config

Comments

Comments are used for adding documentation strings to a JSP page. The two types of comments are the

one which enables documentation to appear in the output from the page.

<!-- comment -->

and the other JSP comments can only be viewed in the original JSP file, or in the source code for the servlet into which the page is translated.

<%-- comment--%>

JSP elements (overview)

- Directives of the form **<%@ ... %>**
- Scripting elements
 - Expressions of the form **<%= expr %>**
 - Scriptlets of the form **<% code %>**
 - Declarations of the form **<%! code %>**
 - JSP Comments **<%-- ... --%>**
- Standard actions
 - Example: **<jsp:useBean> ... </jsp:useBean>**
- Implicit variables like request, response, out

Directives

- these provide global information to the page, for example, import statements, the page for error handling or whether the page is part of a session.
- They have the form

```
<%@ name attribute1="..." , attribute2="..." ... %>
```

page

include

taglib

→ **Include a file at translation time**

→ **Specify page properties**

→ **Specify custom tags**

I. Page directive

The page directive defines a number of page dependent attributes and communicates these to the jsp container

A translation unit (Jsp Source file) can contain more than one instance of the page directive.

All the attributes will apply to the complete translation unit

There shall be only one occurrence of any attribute/value defined by this directive except for the "import" attribute.

Page directive - Syntax

```
<%@ page attribute1="value1" attribute2="value2"  
attribute3=... %>
```

White space after the opening **<%@** and before the closing **%>** is optional, but recommended to improve readability.

Like all JSP tag elements, the page directive supports an XML-based syntax, as follows:

```
<jsp:directive.page  
attribute2="value2".. /> attribute1="value1"
```

Page directive Attributes

There are eleven attributes for the page directive

Attribute	Value	Default	Examples
info	Text string	None	info="Registration form."
language	Scripting language name	"java"	language="java"
contentType	MIME type, character set		contentType="text/html; charset=ISO-8859-1" contentType="text/xml"
extends	Class name	None	extends="com.taglib.wdjsp.MyJspPage"
import	Class and/or package names	None	import="java.net.URL" import="java.util.*, java.text.*"
session	Boolean flag	"true"	session="true"
buffer	Buffer size, or false	"8kb"	buffer="12kb" buffer="false"
autoFlush	Boolean flag	"true"	autoFlush="false"
isThreadSafe	Boolean flag	"true"	isThreadSafe="true"
errorPage	Local URL	None	errorPage="results/failed.jsp"
isErrorPage	Boolean flag	"false"	isErrorPage="false"

Attribute description

Info attribute

The info attribute allows the page author to add a documentation string to the page that summarizes its functionality.

```
<%@ page info="Sample JSP Page directive -  
Sarath." %>
```

Language attribute

The language attribute specifies the scripting language to be used in all scripting elements on the page.

```
<%@ page language="java" %>
```


Attribute description

ContentType attribute

This attribute is used to indicate the MIME type of the response being generated by the JSP page.

`<%@ page contentType="text/xml" %>`

The default MIME type for JSP pages is "text/html".

Extends attribute

The extends attribute identifies the superclass to be used by the JSP container when it is translating the JSP page into a Java servlet

`<%@ page extends="myJspPage" %>`

Attribute description

Import attribute

it extends the set of Java classes which may be referenced in a JSP page.

Importing a Class

```
<%@ page import="java.util.List" %>
```

Importing a Package

```
<%@ page import="java.util.*" %>
```

Importing a Class(es) / Package(s)

```
<%@ page import="java.util.List, java.util.ArrayList,  
java.text.*" %>
```

Attribute description

Session attribute

The session attribute is used to indicate whether or not a JSP page participates in session management

<%@ page session= "false" %>

Buffer attribute

The buffer attribute controls the use of buffered output for a JSP page.

<%@ page buffer="none" %>

<%@ page buffer="14kb" %>

Default value is 8kb

Attribute description

AutoFlush attribute

This attribute is also used for controlling buffered output.

```
<%@ page autoFlush="true" %>
```

IsThreadSafe attribute

The `isThreadSafe` attribute is used to indicate whether your JSP page, once it is compiled into a servlet, is capable of responding to multiple simultaneous requests.

If not, this attribute should be set to false

```
<%@ page isThreadSafe="false" %>
```

Attribute description

ErrorPage attribute

This attribute is used to specify an alternate page to display if an (uncaught) error occurs while the JSP container is processing the page.

```
<%@ page errorPage="/error.jsp" %>
```

IsErrorPage attribute

The isErrorPage attribute is used to mark a JSP page that serves as the error page for one or more other JSP pages.

```
<%@ page isErrorPage="true" %>
```

Scripting elements: expression

- For an expression scripting element like **<%= expr %>**, **expr** is evaluated and the result is converted to a string and placed into the JSP's servlet output stream. In a Java servlet this would be equivalent to

```
PrintWriter out = response.getWriter() ;  
...  
out.print(expr) ;
```

Formats the expression as a string for inclusion in the output of the page

Expression examples

- Displaying request parameters (request is an implicit object available in a JSP)

Your name is `<%= request.getParameter("name") %>`
and your age is `<%= request.getParameter("age") %>`

- Doing calculations

The value of pi is `<%= Math.PI %>` and the square root of two is `<%= Math.sqrt(2.0) %>` and today's date is `<%= new java.util.Date() %>`.

Scripting elements: scriptlet

- For a scriptlet **<% statements %>** the Java statements are placed in the translated servlet's **_jspService** method body (it's like the servlet **service** method which calls either **doGet** or **doPost**)

```
public void _jspService(HttpServletRequest
    request, HttpServletResponse response)
    throws java.io.IOException, ServletException
{ ...
    statements
    ...
}
```


Scriptlet examples

- Check a request parameter

```
<% String name = request.getParameter("name") ;  
    if (name == null)  
    { %>  
        <h3>Please supply a name</h3>  
<% }  
    else  
    { %>  
        <h3>Hello <%= name %></h3>  
<% } %>
```

There are 3 scriptlets here and an expression element

Scripting elements:declaration

- For a declaration **<%! declarations %>** the Java statements are placed in the class outside the **_jspService** method. Typical declarations can be Java instance variable declarations or Java methods

```
// declarations would go here
public void _jspService(...)
{
    ...
}
```

Declaration examples

- For page-wide variable and method declarations.

■ Declaring instance variables

```
<%! private int count = 0; %>  
...  
The count is <%= count++ %>.
```

■ Declaring methods

```
<%!  
private int toInt(String s)  
{  
    return Integer.parseInt(s);  
}  
%>
```

Including files

- Including files at translation time
(when JSP is translated to a servlet)

```
<%@ include file="filename" %>
```

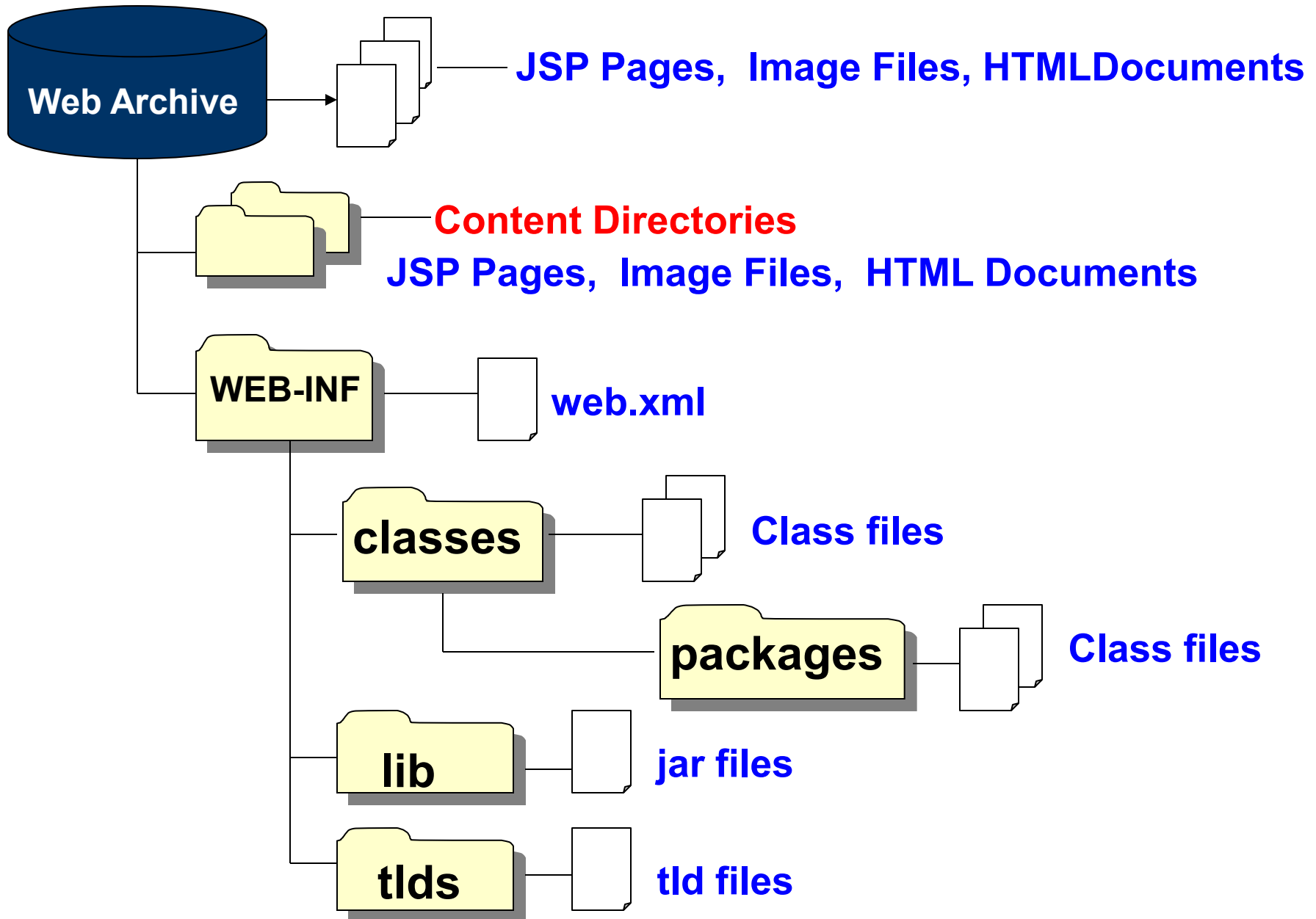
- Including files at request time

```
<jsp:include page="filename" flush = "true" />
```

Where do you put JSP's ?

- If you have a web application called **examples** then create a directory called **jsp** below it and put JSP's there:
- For example a JSP called **test.jsp** would have the path **examples/jsp/test.jsp**
- To run this JSP use the URL
 - **<http://localhost:8080/examples/jsp/test.jsp>**

War Directory Structure



A simple JSP

```
<html>
<head><title>JSP Test</title></head>
<body>
<h1>JSP Test</h1>
Time: <%= new java.util.Date() %>
</body>
</html>
```

The expression scripting element `<%= ... %>` is equivalent to the scriptlet `<% out.print(...); %>`

The implicit out object

- In a scriptlet `<% ... %>` you can use the **out** object to write to the output stream:
- Example:

```
<%  
    out.print("The sum is ");  
    out.print("1 + 2 = " + (1+2));  
%>
```


The implicit request object

■ Example

```
<html>
<head><title>...</title></head>
<body>
<h1>...</h1>
<p><%= request.getParameter("greeting") %></p>
</body></html>
```

Try this using

[http://localhost:8080/examples/jsp/simple/
greeting0.jsp?greeting=Hello Frank](http://localhost:8080/examples/jsp/simple/greeting0.jsp?greeting=Hello%20Frank)

Implicit Objects	Description
<code>request</code>	The client's request. This is usually a subclass of <code>HttpServletRequest</code> . This has the parameter list if there is one.
<code>response</code>	The JSP page's response, a subclass of <code>HttpServletResponse</code> .
<code>pageContext</code>	Page attributes and implicit objects (essentially what makes up the server environment in which the JSP runs) need to be accessible through a uniform API, to allow the JSP engine to compile pages. But each server will have specific implementations of these attributes and objects. The solution to this problem is for the JSP engine to compile in code that uses a factory class to return the server's implementation of the <code>PageContext</code> class. That <code>PageContext</code> class has been initialized with the <code>request</code> and <code>response</code> objects and some of the attributes from the page directive (<code>errorpage</code>, <code>session</code>, <code>buffer</code> and <code>autoflush</code>) and provides the other implicit objects for the page request. We'll see more on this in a moment.
<code>session</code>	The HTTP session object associated with the request.
<code>application</code>	The servlet context returned by a call to <code>getServletConfig().getContext()</code> (see Chapter 5).
<code>out</code>	The object representing the output stream.

Processing form using GET

```
<html>
<head><title>JSP Processing ...</title></head>
<body>
<h1>JSP Processing form with GET</h1>
<form action="doForm1.jsp" method="GET">
First name: <input type="text" name="firstName"><br />
Last name: <input type="text" name="lastName">
<p><input type="submit" name="button"
        value="SubmitName"></p>
</form>
</body>
</html>
```

examples/jsp/forms/form1_get.html

Processing form using POST

```
<html>
<head><title>JSP Processing ...</title></head>
<body>
<h1>JSP Processing form with POST</h1>
<form action="doForm1.jsp" method="POST">
First name: <input type="text" name="firstName"><br />
Last name: <input type="text" name="lastName">
<p><input type="submit" name="button"
      value="SubmitName"></p>
</form>
</body>
</html>
```

examples/jsp/forms/form1_post.html

doForm1.jsp

```
<%@ include file="../../../doctype.html" %>
<head>
  <title>JSP Form Results</title>
</head>
<body>
<h1>JSP Form Results</h1>
Hello <%= request.getParameter("firstName") %>
<%= request.getParameter("lastName") %>
</body>
</html>
```

examples/jsp/forms/doForm1.jsp

Try this using

http://localhost:8080/examples/jsp/forms/form1_get.html

Temperature conversion

Fahrenheit to Celsius Conversion

Fahrenheit temperature:

Convert to Celsius

input

Fahrenheit to Celsius Conversion

100.00F is 37.78C

[Another conversion](#)

output

temperature.jsp (1)

```
<%@ page import="java.text.DecimalFormat" %>
<html>
<head><title>Fahrenheit ... Conversion</title></head>
<body>
<h1>Fahrenheit to Celsius Conversion</h1>
<% String self = request.getRequestURI();
    if (request.getParameter("convert") == null)
    {
%>
    <form action="<%= self %>" method="POST">
    Fahrenheit temperature:
    <input type="text" name="fahrenheit" />
    <p><input type="submit" name="convert"
        value="Convert to Celsius" /></p>
    </form>
```

temperature.jsp (2)

```
<%  
    }  
    else  
    {  
        double fahr = 0.0;  
        try  
        {  
            fahr = Double.parseDouble(  
                request.getParameter("fahrenheit"));  
        }  
        catch (NumberFormatException e)  
        {  
            // do nothing, accept default value  
        }  
    }  
}
```


temperature.jsp (3)

```
double celsius = (fahr - 32.0) * (5.0/9.0);
DecimalFormat f2 = new DecimalFormat("0.00");

%>

<%= f2.format(fahr) %>F is
<%= f2.format(celsius) %>C
<p><a href="<%= self %>">Another conversion</a>
</p>

<%
    }
%>
</body>
</html>
```

Java Beans

- Special classes that encapsulate some data
- They have a default constructor
- get and set methods for data fields (properties)
- A bean can be constructed in JSP using
 - `<jsp:useBean id = "... " class = "... " />`
- If the bean already exists this statement does nothing

setting properties

- To set a property of a bean use

```
<jsp:setProperty name="..."  
    property="..." value="..." />
```

- To set a property using the value of a request parameter use

```
<jsp:setProperty name="..."  
    property="..." param="..." />
```

getting properties

- To get a property of a bean use

```
<jsp:getProperty name="..."  
    property="..." />
```

useBean Attributes	Description
<code>id="name"</code>	The name by which the instance of the bean can be referenced in the page. Other Bean tags use <code>name</code> to refer to it, so do note the difference.
<code>scope="page"</code>	The scope over which the bean can be called. More below.
<code>class="package.class"</code>	Fully qualified name of the bean class. Because this needs to be the full name, you don't need to <code>import</code> these classes in the <code>page</code> directive.
<code>beanName="name"</code>	This allows you to specify a serialized bean (<code>.ser</code> file) or a bean's name that can be passed to the <code>instantiate()</code> method from the <code>java.beans.Beans</code> . Needs an associated <code>type</code> tag rather than <code>class</code> .
<code>type="package.class"</code>	A synonym for <code>class</code> that is used for <code>beanName</code> .

A Greeting bean

```
package beans;
public class Greeting
{
    private String greeting; // the property

    public Greeting()
    { greeting = "Hello World"; }

    public String getGreeting()
    { return greeting; }

    public void setGreeting(String g)
    { greeting = (g == null) ? "Hello World" : g;
    }
}
```

[beans/Greeting.java](#)

Naming convention

- If the property name is **greeting**
- the get method must have the name **getGreeting**
- the set method must have the name **setGreeting**

Creating a Greeting bean (1)

- Create a bean and use default property

```
<jsp:useBean id="hello" class="beans.Greeting" />
```

- Create a bean and set its property when it is constructed

```
<jsp:useBean id="hello" class="beans.Greeting" >  
  <jsp:setProperty name="hello" property="greeting"  
    value="Hello JSP World" />  
</jsp:useBean>
```

- Here `<jsp:setProperty>` is in the body of the `<jsp:useBean>` element.

Creating a Greeting bean (2)

- Create a bean and set its property after it has been constructed

```
<jsp:useBean id="hello" class="beans.Greeting" />  
<jsp:setProperty name="hello" property="greeting"  
    value="Hello JSP World" />
```

- The `<jsp:setProperty>` tag is now outside the `<jsp:useBean>` tag so it will always set the property, not just when the bean is constructed

greeting1.jsp

```
<jsp:useBean id="hello" class="beans.Greeting" />
<jsp:setProperty name="hello" property="greeting"
    value="Hello JSP World" />
<html>
<head>
<title>Greeting JSP that uses a Greeting bean</title>
</head>
<body>
<h1>Greeting JSP that uses a Greeting bean</h1>
<p><jsp:getProperty name="hello" property="greeting" />
</p>
</body>
</html>
```

[test/jsp/greeting1.jsp](http://localhost:8080/test/jsp/greeting1.jsp)

<http://localhost:8080/test/jsp/greeting1.jsp>

Two beans

One initialized explicitly
and the other is initialized
using a request parameter

greeting2.jsp

```
<jsp:useBean id="greet1" class="beans.Greeting" />
<jsp:useBean id="greet2" class="beans.Greeting" />
<jsp:setProperty name="greet1" property="greeting"
    value="Hello JSP World" />
<jsp:setProperty name="greet2" property="greeting"
    param="greeting" />
<html><head><title>Greeting JSP using two Greeting
beans</title></head><body>
<h1>Greeting JSP using two Greeting beans</h1>
<p>1st bean: <jsp:getProperty name="greet1"
    property="greeting" /></p>
<p>2nd bean: <jsp:getProperty name="greet2"
    property="greeting" /></p></body></html>
```

[test/jsp/greeting2.jsp](http://localhost:8080/test/jsp/greeting2.jsp) <http://localhost:8080/test/jsp/greeting2.jsp>

<http://localhost:8080/test/jsp/greeting2.jsp?greeting=Hello+friend>

three beans and include file

One initialized explicitly, one
is
initialized using a request
parameter, and one is
initialized using `getParameter`

greeting3.jsp (1)

```
<jsp:useBean id="greet1" class="beans.Greeting" />
<jsp:useBean id="greet2" class="beans.Greeting" />
<jsp:useBean id="greet3" class="beans.Greeting" />
<jsp:setProperty name="greet1" property="greeting"
    value="Hello JSP World" />
<jsp:setProperty name="greet2" property="greeting"
    param="greeting" />

<%-- Following works but param method is better --%>

<jsp:setProperty name="greet3" property="greeting"
    value="<%= request.getParameter(\"greeting\") %>" />
```

greeting3.jsp (2)

```
<%-- Include file contains doctype and html tag --%>
<jsp:include page="doctype.html" flush="true" />
<head><title>Greeting JSP</title></head><body>
<h1>Greeting JSP</h1>
<p>1st bean: <jsp:getProperty name="greet1"
               property="greeting" /></p>
<p>2nd bean: <jsp:getProperty name="greet2"
               property="greeting" /></p>
<p>3rd bean: <jsp:getProperty name="greet3"
               property="greeting" /></p>
<p>request: <%= request.getParameter("greeting") %></p>
</body></html>
```

[test/jsp/greeting3.jsp](http://localhost:8080/test/jsp/greeting3.jsp) <http://localhost:8080/test/jsp/greeting3.jsp>

<http://localhost:8080/test/jsp/greeting3.jsp?greeting=Hello+Fred>

Beans and forms

Using a bean to represent the
data of a form

form1_bean.html (1)

```
<html>
<head>
<title>JSP Processing form using a bean</title>
</head>
<body>
<h1>JSP Processing form using a bean</h1>
<form action="doForm1Bean.jsp" method="GET">
First name: <input type="text" name="firstName"></ br>
Last name: <input type="text" name="lastName">
<p><input type="submit" name="button"
      value="Submit Name"></p>
</form>
</body>
</html>
```

test/jsp/forms/form1_bean.html

http://localhost:8080/test/jsp/forms/form1_bean.html

A Name bean

```
package beans;
public class NameBean
{
    private String firstName; // property
    private String lastName;  // property
    public String getFirstName()
    { return firstName; }
    public String getLastName()
    { return lastName; }

    public void setFirstName(String first)
    { firstName = first; }
    public void setLastName(String last)
    { lastName = last; }
}
```

[beans/NameBean.java](#)

doForm1Bean.jsp

```
<jsp:useBean id="name" class="beans.NameBean" />
<jsp:setProperty name="name" property="*" />
<jsp:include page="../doctype.html" flush="true" />
<head>
<title>JSP Form results using a bean</title>
</head>
<body>
<h1>JSP Form results using a bean</h1>
Hello
<jsp:getProperty name="name" property="firstName" />
<jsp:getProperty name="name" property="lastName" />
</body>
</html>
```

<test/jsp/forms/doForm1Bean.jsp>

Note the use of `property="*"` to set all the bean properties using the form data values

MVC approach

- MVC stands for Model-View-Controller
- Controller:
 - controls the application by getting the input and forwarding the request to the appropriate page
- View:
 - visual appearance of the output data
- Model:
 - logic, calculations necessary to produce the data. Sometimes called business logic

MVC example (Temperature)

- Getting the input (Fahrenheit temperature)
 - `controller.jsp` (dispatch (forward) request)
 - `form.jsp` (user input of temperature)
- Displaying the output (Celsius temperature)
 - `response.jsp`
- Model (logic and calculations done by beans)
 - `TemperatureBean.java` (form data)
 - `TemperatureCalculatorBean.java`
(does the temperature conversio)

TemperatureBean

```
package beans;  
public class TemperatureBean  
{  
    private String temp;  
  
    public String getTemperature()  
    { return temp; }  
  
    public void setTemperature(String t)  
    { temp = t; }  
}
```

[beans/TemperatureBean.java](#)

TemperatureCalculatorBean

```
package beans;  
import java.text.DecimalFormat;  
public class TemperatureCalculatorBean  
{  
    private double celsius;  
  
    public String getTemperature()  
    {  
        return  
            new DecimalFormat("0.00").format(celsius);  
    }  
  
    // continued on next slide
```

TemperatureCalculatorBean

```
public void setTemperature(String fahr)
{
    double f;
    try
    {
        f = Double.parseDouble(fahr);
    }
    catch (NumberFormatException e)
    {
        f = 0.0;
    }
    celsius = (f - 32.0) * (5.0 / 9.0);
}
```

[beans/TemperatureCalculatorBean.java](#)

Forwarding a page

- A JSP can forward the request to another page using
 - `<jsp:forward page="file.jsp" />`
- The important idea here is that this new page receives all the request parameters of the forwarding page.

controller.jsp

```
<jsp:useBean id="temp" class="beans.TemperatureBean">
<jsp:setProperty name="temp" property="*" />
</jsp:useBean>
<% if (request.getParameter("temperature") == null)
{
%>
    <jsp:forward page="form.jsp" />
<%
} else {
%>
    <jsp:forward page="response.jsp" />
<% }
%>
```

[test/jsp/temperature/controller.jsp](http://localhost:8080/test/jsp/temperature/controller.jsp)

<http://localhost:8080/test/jsp/temperature/controller.jsp>

form.jsp

```
<jsp:include page="../../../doctype.html" flush="true" />
<head>
<title>Fahrenheit to Celsius Conversion</title>
</head>
<body>
<h1>Fahrenheit to Celsius Conversion</h1>
<form action="controller.jsp" method="GET">
Fahrenheit temperature:
<input type="text" name="temperature"></ br>
<input type="submit" name="button"
    value="Submit Temperature"></p>
</form>
</body>
</html>
```

<test/jsp/temperature/form.jsp>

response.jsp

```
<jsp:useBean id="temp" class="beans.Temperature"
  scope="request" />
<jsp:useBean id="convert"
  class="beans.TempertureCalculatorBean" />
<jsp:setProperty name="convert" property="temperature"
  value="<%= temp.getTemperature() %>" />
<jsp:include page=" ../doctype.html" flush="true" />
<head><title>Fahrenheit to Celsius Conversion</title>
</head><body><h1>Fahrenheit to Celsius Conversion</h1>
<jsp:getProperty name="temp" property="temperature" />
Fahrenheit is
<jsp:getProperty name="convert"
property="temperature" /> Celsius.
<p><a href="controller.jsp">Convert another .. </a>
</p></body></html>
```

<test/jsp/temperature/response.jsp>

Databases using JSP

Display the books database
table

bookdisplay1.jsp

BookBean.java

BookDataBean.java

DBConnectorBean.java

bookdisplay1.jsp (1)

```
<%@ page import="java.text.*, java.util.*, beans.*" %>
<jsp:useBean id="db" class="DBConnectorBean" >
    <jsp:setProperty name="db" property="driver"
        value="<%= application.getInitParameter(
                                \"jdbc.driver\") %>"/>
    <jsp:setProperty name="db" property="url"
        value="<%= application.getInitParameter(
                                \"jdbc.url\") %>"/>
    <jsp:setProperty name="db" property="username"
        value="<%= application.getInitParameter(
                                \"jdbc.username\") %>"/>
    <jsp:setProperty name="db" property="password"
        value="<%= application.getInitParameter(
                                \"jdbc.password\") %>"/>
</jsp:useBean>
```

bookdisplay1.jsp (2)

```
<jsp:useBean id="book" class="BookBean" />

<jsp:useBean id="bookDataBean" class="BookDataBean">
    <jsp:setProperty name="bookDataBean"
        property="connection"
        value="<%= db.getConnection() %>" />
</jsp:useBean>

<%! DecimalFormat cents = new DecimalFormat("0.00"); %>

<jsp:include page="doctype.html" flush="true" />
<head><title>Displaying the books database table
</title></head>
<body>
<h1>Displaying the books database table</h1>
```

bookdisplay1.jsp (3)

```
<table border="1">
<tr>
  <th>ISBN</th><th>TITLE</th><th>AUTHOR</th>
  <th>PUB</th><th>YEAR</th><th>PRICE</th>
</tr>

<%
  Iterator bookListIterator =
    bookDataBean.getIterator();
  while (bookListIterator.hasNext())
  {
    book = (BookBean) bookListIterator.next();
  }
%>
```


bookdisplay1.jsp (4)

```
<tr>
<td><%= book.getIsbn() %></td>
<td><%= book.getTitle() %></td>
<td><%= book.getAuthor() %></td>
<td><%= book.getPub() %></td>
<td><%= book.getYear() %></td>
<td><%= cents.format(book.getPrice()) %></td>
</tr>

<%
    }
%>

</table></body></html>
```

<test/jsp/bookdisplay1.jsp>

<http://localhost:8080/test/jsp/bookdisplay1.jsp>